

Interface DISPO ↔ Backend

1 Purpose

This document defines the REST API contract between the external DISPO system and the Rückbestätigungs-Backend.

The DISPO system remains the leading system for order data and delivery planning. The backend does not create business orders independently. It receives confirmation-relevant order data, stores a local snapshot, creates confirmation requests, sends customer notifications and manages confirmation status.

2 System Boundary

2.1 DISPO System Responsibilities

- managing existing orders
- delivery planning
- assigning delivery dates and delivery windows
- dispatcher workflow
- manual start of confirmation process

2.2 Backend Responsibilities

- receiving confirmation requests
- validating incoming order data
- storing local order snapshots
- creating confirmation requests
- generating token-based action URLs
- sending customer notification e-mails
- managing confirmation status
- storing customer responses
- handling expiration after 24 hours

3 Process Start

The dispatcher manually starts the process inside DISPO:

Rückbestätigung senden

DISPO then calls the backend via REST API.

4 REST Endpoint

4.1 Create Confirmation Request

POST /api/dispo/confirmation-requests

This endpoint creates a confirmation request and triggers synchronous e-mail sending.

5 Request DTO

5.1 Request JSON

```
{
  "externalOrderId": "A-1024",
  "customerName": "Max Muller",
  "customerEmail": "daniel@example.com",
  "deliveryAddress": "Beispielstrasse 12, 97070
    Wurzburg",
  "product": "Heizol",
  "quantityLiters": 3000,
  "deliveryDate": "2026-06-12",
  "deliveryWindowStart": "10:00",
  "deliveryWindowEnd": "11:00"
}
```

5.2 Field Description

Field	Type	Required	Description
externalOrderId	string	yes	Order identifier from DISPO
customerName	string	yes	Customer name
customerEmail	string	yes	E-mail address for confirmation
deliveryAddress	string	yes	Full delivery address
product	string	yes	Product to deliver
quantityLiters	integer	yes	Ordered quantity in liters
deliveryDate	date	yes	ISO date YYYY-MM-DD
deliveryWindowStart	time	yes	HH:mm
deliveryWindowEnd	time	yes	HH:mm

5.3 Fields intentionally excluded in v1

- externalTourId
- customerNumber
- pricePer100Liters
- dispatcherId
- dispatcherName

6 Success Response

6.1 201 Created

Returned only if:

- request validation succeeded
- confirmation request was created internally
- token URLs were generated internally
- e-mail was sent successfully
- status was set to SENT

```
{
  "externalOrderId": "A-1024",
  "confirmationStatus": "SENT"
}
```

6.2 Response Fields

Field	Type	Description
externalOrderId	string	DISPO order identifier used for correlation
confirmationStatus	string	Current confirmation status, normally SENT

6.3 Internal Fields not exposed in v1

The backend internally creates and stores confirmation requests. However, the internal confirmation request id is not exposed to DISPO in API v1.

Internally generated but not returned:

- confirmationRequestId
- confirmActionUrl
- rejectActionUrl
- sentAt
- expiresAt

7 Confirmation Status Values

API Value	German Label	Meaning
SENT	versendet	Waiting for customer response
CONFIRMED	bestätigt	Customer accepted delivery window
REJECTED	abgelehnt	Customer rejected delivery window
NO_RESPONSE	keine Rückmeldung	No valid response within 24h

7.1 Design Decision: No confirmationRequestId in DISPO API v1

The DISPO-facing API communicates on order level.

DISPO identifies the order through externalOrderId. Individual confirmation requests, token handling, active request tracking, old request invalidation and expiration handling are managed internally by the backend.

Therefore, confirmationRequestId is not exposed to DISPO in API v1.

This avoids coupling DISPO to the internal data model of the confirmation backend.

The backend is responsible for ensuring that only the latest active confirmation request can update the current confirmation status of an order.

8 Validation Rules

Rule ID	Field	Rule
VAL-001	externalOrderId	must not be blank
VAL-002	customerName	must not be blank
VAL-003	customerEmail	must not be blank and must be a valid e-mail address
VAL-004	deliveryAddress	must not be blank
VAL-005	product	must not be blank
VAL-006	quantityLiters	must be greater than 0
VAL-007	deliveryDate	must be a valid date in ISO format YYYY-MM-DD
VAL-008	deliveryWindowStart	must be a valid time in format HH:mm
VAL-009	deliveryWindowEnd	must be a valid time in format HH:mm
VAL-010	deliveryWindow	deliveryWindowStart must be before deliveryWindowEnd

9 Error Responses

All error responses use the same structure.

<pre>{ "code": "VALIDATION_ERROR", "message": "Human-readable error message", "status": 400, "path": "/api/dispo/confirmation-requests", "timestamp": "2026-06-12T08:15:00Z" }</pre>
--

9.1 Fields

Field	Type	Description
code	string	Stable machine-readable error code
message	string	Human-readable error description
status	integer	HTTP status code
path	string	Request path where the error occurred
timestamp	datetime	Time when the error occurred

9.2 400 Validation Error

Returned when the request contains invalid or missing data.

400 Bad Request

```
{
  "code": "VALIDATION_ERROR",
  "message": "Customer e-mail is required for digital
             confirmation.",
  "status": 400,
  "path": "/api/dispo/confirmation-requests",
  "timestamp": "2026-06-12T08:15:00Z"
}
```

9.3 502 E-Mail Sending Failed

Returned when the confirmation e-mail could not be sent.

502 Bad Gateway

```
{
  "code": "EMAIL_SENDING_FAILED",
  "message": "The confirmation e-mail could not be sent.",
  "status": 502,
  "path": "/api/dispo/confirmation-requests",
  "timestamp": "2026-06-12T08:15:00Z"
}
```

9.4 500 Internal Error

Returned when an unexpected technical error occurs.

500 Internal Server Error

```
{
  "code": "INTERNAL_ERROR",
  "message": "An unexpected technical error occurred.",
  "status": 500,
  "path": "/api/dispo/confirmation-requests",
  "timestamp": "2026-06-12T08:15:00Z"
}
```

10 Business Rules

Rule ID	Rule
BR-001	The order already exists in DISPO before the confirmation process starts.
BR-002	DISPO is the leading system for order data and delivery planning.
BR-003	The backend stores a local snapshot only when DISPO starts a confirmation request.
BR-004	The confirmation process is started manually by the dispatcher in DISPO.

BR-005	DISPO calls the backend directly via REST API.
BR-006	A confirmation request always refers to exactly one order and one concrete delivery window.
BR-007	Tour data is not part of API v1.
BR-008	The first successful confirmation status returned by the backend is SENT.
BR-009	SENT is set only after successful technical e-mail sending.
BR-010	The 24-hour response timer starts only after successful technical e-mail sending.
BR-011	If e-mail sending fails, the API returns an error and no successful confirmation status is created.
BR-012	The e-mail contains token-based action buttons.
BR-013	Action links open a minimal customer confirmation page.
BR-014	Opening an action link does not directly change the confirmation status.
BR-015	The confirmation status is changed only after a final submit on the customer page.
BR-016	Only one valid customer response is allowed per confirmation request.
BR-017	Only the latest active request can update the current confirmation status.
BR-018	Old, inactive or replaced links must not update the current order status.
BR-019	Customer comments are optional.
BR-020	Customer comments are stored but not automatically interpreted.
BR-021	The system does not perform automatic tour planning, route optimization or replanning.
BR-022	If the customer rejects or does not respond, the dispatcher decides manually how to proceed.
BR-023	When the status changes to CONFIRMED, REJECTED or NO_RESPONSE, the backend sends a callback to DISPO.
BR-024	If DISPO sends the same request for the same externalOrderId with unchanged confirmation-relevant data, the backend treats it as duplicate and does not send a second e-mail.
BR-025	If DISPO sends a request for the same externalOrderId with changed confirmation-relevant data, the backend invalidates previous active requests and creates a new confirmation request.

11 Duplicate Handling

DISPO may send more than one request for the same externalOrderId.

The backend distinguishes between duplicate requests with unchanged confirmation data and new requests with changed confirmation-relevant data.

11.1 Duplicate with unchanged data

If DISPO sends the same request again and the confirmation-relevant data has not changed, the backend treats the request as duplicate.

Confirmation-relevant data includes:

- customerEmail

- deliveryAddress
- product
- quantityLiters
- deliveryDate
- deliveryWindowStart
- deliveryWindowEnd

In this case:

- no new confirmation request is created
- no second e-mail is sent
- the existing active confirmation request is returned

200 OK

```
{
  "externalOrderId": "A-1024",
  "confirmationStatus": "SENT"
}
```

11.2 Changed confirmation-relevant data

If DISPO sends a request for the same externalOrderId but confirmation-relevant data has changed, the backend treats it as a new confirmation request.

In this case:

- previous active requests are invalidated internally
- a new confirmation request is created
- a new e-mail is sent
- the new confirmation request is returned

201 Created

```
{
  "externalOrderId": "A-1024",
  "confirmationStatus": "SENT"
}
```

The API v1 does not return 409 Conflict for duplicate active requests.

12 Sequence Overview

12.1 Successful Flow

```
DISPO -> Backend -> Notification Service -> Backend ->
DISPO
```

12.2 Failed E-Mail Flow

<pre>DISPO -> Backend -> Notification Service (fail) -> Backend -> 502</pre>
--

13 Assumptions

- DISPO can call backend via REST
- e-mail is MVP channel
- sending is synchronous
- expiration is 24h

14 Open Questions

- Which exact fields does real DISPO provide?
- Should callback retries be implemented?
- Should reminder logic be added?